HPC Cluster

Ivan Sen
Computer Science
UCU
Lviv, Ukraine
sen.pn@ucu.edu.ua

Vladyslav Sydorak
Computer Science
UCU
Lviv, Ukraine
sydorak.pn@ucu.edu.ua

Taras Patsahan
Institute for
Condensed Matter Physics
Lviv, Ukraine

Oleh Fareniuk
Institute for
Condensed Matter Physics
UCU
Lviv, Ukraine

I. Introduction

Parallel computing is essential for performing time-consuming, massive tasks. For this purpose, multiprocessor/multi-core computers and GPU[1] video accelerators are used. However, when a computational task goes beyond the capacity of a single computer, it becomes necessary to combine computers into a single computing resource - a so-called cluster. Computing clusters have been actively used for many years to solve scientific and applied problems. They are the basis for the creation of huge computing centers that provide services to both academic institutions and private companies. Computing clusters are often installed separately in universities, research institutes and laboratories, small companies and industrial facilities. These are relatively small clusters that aim to serve a limited number of users (< 50 active users) and consist of < 30 nodes. Usually, such clusters do not require any extraordinary infrastructure and are supported by 1-2 employees.

An important stage is the configuration of the cluster. Of course, it starts with design, i.e. determining the key cluster nodes (e.g. master server, file server, computing nodes, etc.), their interconnection via the network, selection of appropriate hardware and software.

The main objective of this project is to build a working prototype of a computing cluster on several nodes (3-4 nodes) using the current software designed for clusters. The structure of the cluster is simple: a coordinating node, computing nodes and a file server. All these elements are located in a closed local area network, and the coordinating node has an additional Internet connection through which users log in to the cluster. The user should be able to run their tasks from the coordinating node and monitor their progress there. OpenMPI[2] support is required to run parallel programs. Tasks are launched and managed using the SLURM[3] scheduler. The cluster must be installed on the Rocky Linux operating system and OpenHPC components.

II. Possible software choices

Because of long legacy in a sphere of Beowulf[4] clusters, there are many options to consider. As a base for our software we had to choose between Rocks Cluster and OpenHPC. Rocks Cluster, while being legacy software, is still a solid option to consider. It is a fully self-contained piece of software, based around CentOS 7, which went EOL[5] on June 30, 2024.

OpenHPC is a collection of useful packages, not a single operating system. It contains packages that include provisioning tools, resource management systems, Input/Output clients, development tools and different scientific libraries. We decided to go with Open HPC, because it is still being developed and improved, so for the long term it is a better solution.

For managing tasks, there are different options, each having their pros and cons, the most popular being PBS(Portable Batch System) and SLURM (also known as Simple Linux Utility for Resource Management). We finally decided to use SLURM, because it's easier to setup than PBS, while being more widely used in the industry.

For provisioning and managing cluster nodes, Warewulf was employed. Warewulf is a cluster management toolkit that levarages iPXE[6] to enable the deployment, configuration, and management of operating systems on compute nodes without requiring local disks on each node. This simplifies the process of node provisioning and ensures efficient resource utilization within the virtualised cluster.

III. VM-Based Deployment

For the purpose of testing and development, a virtualized computing cluster was deployed using the Proxmox hypervisor. Proxmox is an open-source virtualization platform that combines Kernel-based Virtual Machine and Linux Containers with a web-based interface for managing virtual machines and containers. It is widely used for enterprise-grade virtualization and provides features such as high availability, live migration, and backup solutions.

Proxmox allowed the creation and management of three virtual machines, simulating a small-scale cluster environment.

[1]Graphics Processing Unit.

[2]Open MPI is an open source implementation of the Message Passing Interface specification.

[3]The Slurm Workload Manager, formerly known as Simple Linux Utility for Resource Management, is a free and open-source job scheduler for Linux and Unix-like kernels, used by many of the world's supercomputers and computer clusters.

[4]Beowulf Clusters are scalable performance clusters based on commodity hardware, on a private system network, with open source software (Linux) infrastructure.

[5]EOL stands for "End of Life," which refers to the point in a product's lifecycle when it no longer receives support or updates from the manufacturer.

[6]iPXE(Improved Preboot eXecution Environment) is an open-source boot firmware that extends PXE.

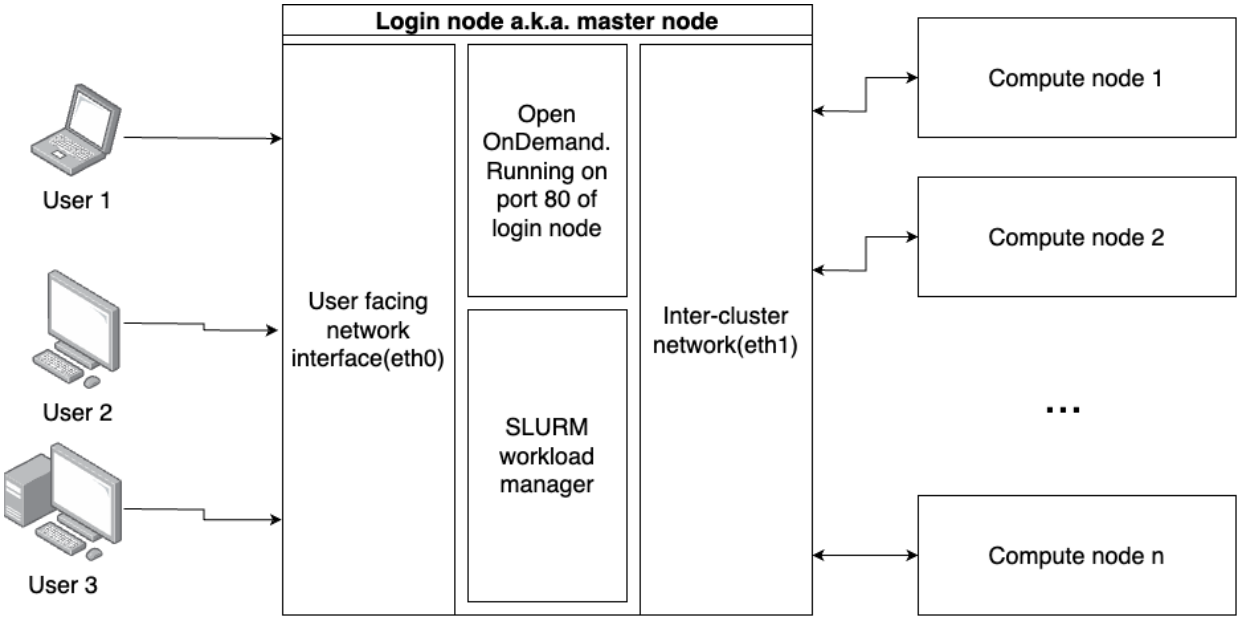


Fig. 1. Test cluster network scheme

The setup consisted of a single coordinating node, responsible for managing the cluster, and two compute nodes that executed the parallel computations. Number of compute nodes can be expanded very easily. Inner cluster network is isolated from the wide network, the only entry point being Master Node.

One of the key advantages of using virtualization for cluster simulation is the ability to create an isolated, high-speed internal network between nodes. The virtual network within the cluster can achieve transfer speeds that are limited only by the processing power of the host machine. In this case, file transfers within the cluster reached speeds of up to 5 Gbps, providing an efficient environment for testing distributed computing tasks and communication protocols.

The cluster was configured using Rocky Linux, a free and open-source Linux distribution that is ideally suited for server environments. It offers long-term stability and enterprise-grade reliability, while also being open-source and community driven, making it a suitable choice for cluster deployments.

Additionally, virtualised deployment provides flexibility in managing and configuring the cluster. It simplifies testing different software configurations and allows rapid provisioning of new nodes as needed. This makes it an ideal choice for developing and validating cluster configurations before transitioning to a physical hardware implementation.

IV. Ansible

Ansible is an open-source automation platform that simplifies complex infrastructure management through declarative, YAML[7]-based playbooks. It was chosen to orchestrate the entire software installation workflow by executing a playbook that configures the master node, installs OpenHPC components, sets up the SLURM resource management system, and configures Warewulf for compute node provisioning.

Agentless architecture[8] is particularly advantageous for cluster deployments as it eliminates the need to install and maintain client software on managed nodes, which simplifies the initial setup process. For this specific project, Ansible's push-based execution model works particularly well with the master-compute node architecture, allowing for controlled and predictable provisioning from the coordinating node.

The module library that Ansible provides allows for direct interaction with various systems required in the OpenHPC stack, including package managers, system services, and specialized components. The playbook uses the `ansible.builtin.dnf` package manager. It manages system services through the `ansible.builtin.systemd_service` module, handles time synchronization with `chrony`, and uses `warewulfd` for node provisioning, `munge` for authentication, and `slurmctld / slurmd` for job scheduling. File manipulation, network configuration, system control, text editing, and command execution are handled via the `ansible.builtin` module. And this is only a small part of the full feature list.

V. Open OnDemand

Open OnDemand is an open-source, web portal developed by OSC[9]. It provides users with intuitive and easier to understand interface to acsess HPC resources. Using it users can

[7]Yet Another Markup Language. Widely used format for writing configuration files for different DevOps tools, programs, and applications because of its human-readable and intuitive syntax.

[8]Meaning it manages remote systems using standard protocols like SSH without requiring any software agent to be installed and running on target machines.

[9]Ohio Supercomputer Center.

deploy jobs on a cluster, but they also can work with interactive apps which include such applications as VSCode server, RStudio server and Jupyter server. While it eliminates the need to use SSH[10]. It also simplifies work of the system administrators using features like: automatic mapping of emails to linux users, maintance mode and different diagnostic tools.

VI. Bare metal



Fig. 2. Baremetal test cluster

A. Specs

The cluster (see Fig. 2) consists of three SuperMicro 1U servers. Each server is equipped with:

- 2× Intel Xeon E5-2620v2 (6 cores/12 threads)
- 48 GB DDR3 memory
- 500 GB Western Digital Black HDD (only used on the master node)

A D-Link 100 Mb/s L3 networking appliance is currently used for testing, with plans to migrate to a 1 Gb/s L2 switch.[11]

B. Problems and Fixes

Below is a comprehensive list of the major issues encountered during setup:

- Due to the old hardware, default iPXE images of the warewulf, did not boot. This was resolved by rebuilding the boot images, and rewriting the previous configuration to serve new boot files.
- The external network was mistakenly connected to the inter-cluster network. Significant time was spent debugging until the issue was resolved by unplugging the cable.
- The network operates correctly with a router but fails when using a switch. A virtual network configuration was simulated, consisting of a master node functioning as a DHCP server and two compute nodes, all interconnected via a Layer 2 switch. The simulation, conducted using Cisco Packet Tracer[12], demonstrated that such a setup can operate effectively without the need for a router.
- The DHCP[13] server fails to start after a reboot because the network interface loses its IP address. A planned fix involves implementing a one-shot `systemd` script to restore the interface IP after reboot.

Numerous minor issues were also encountered, primarily due to human error, but are omitted here to keep this report concise.

C. Benchmarks

This section presents MPI benchmarks for the cluster, along with analysis of the results. Intel MPI Benchmarks, compiled for OpenMPI version 5.0, were used to evaluate network performance in terms of speed and latency from an MPI usage perspective. Benchmark results are shown in Figures 3–6.

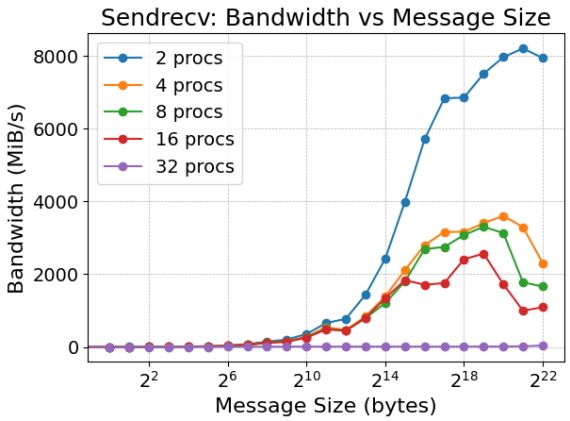


Fig. 3.

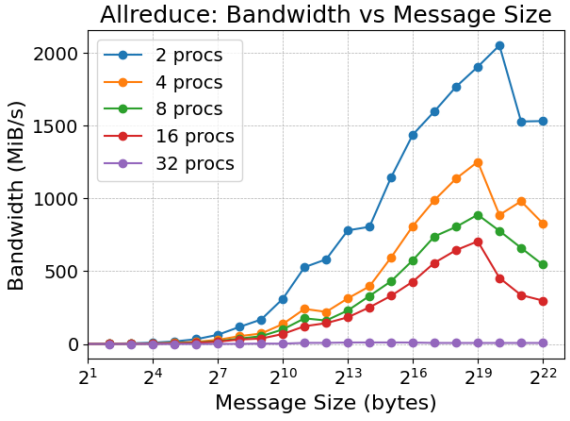


Fig. 4.

[10]Secure SHell networking protocol commomly used to acsess shell remotely.

[11]A layer-2 device generally offers higher throughput, is usually less expensive, and is easier to configure.

[12]Cisco Packet Tracer, a network simulation tool that allows to design, configure, and troubleshoot virtual networks.

[13]Dynamic Host Configuration Protocol, a network management protocol that automatically assigns IP addresses and other configuration settings to devices on a network.

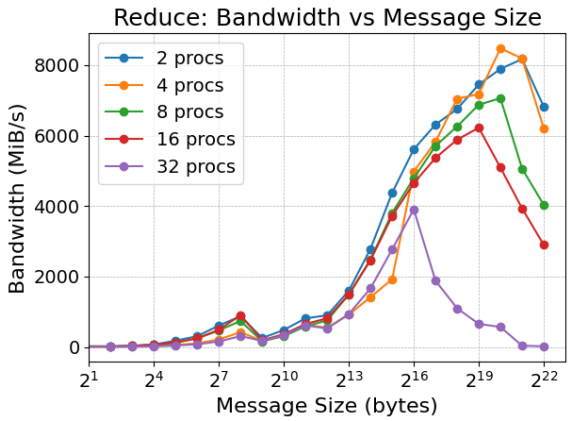


Fig. 5.

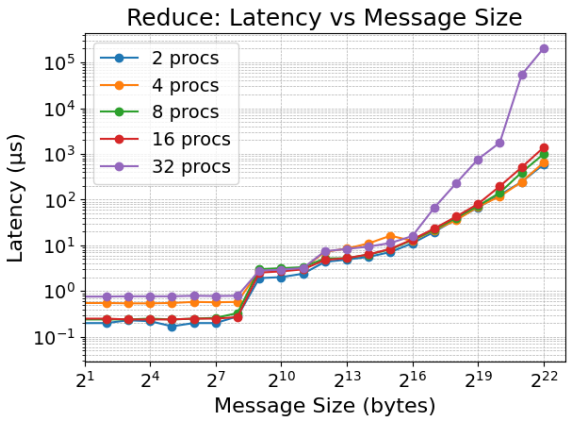


Fig. 6.

The results support several general conclusions:

- As message size increases, both latency and bandwidth rise up to a certain point, after which bandwidth plateaus. Larger messages reduce contention, but communication requires more time, explaining this trend.
- As the number of processes increases, latency increases and per-process bandwidth decreases. Given fixed total throughput, each process receives a smaller share of the available bandwidth.

A notable outlier is the Reduce latency graph: latency remains consistent across most process counts, with only the 32-process case deviating slightly. This occurs because average time changes only minimally with increasing message size, aside from specific milestones or jumps. These subtle variations are difficult to observe on a logarithmic scale.

VII. Conclusions

This project demonstrates the successful construction of an OHPC cluster, beginning with virtual machines and the default PDF guide, and ending with in a fully functional bare-metal cluster with an automated setup using Ansible playbooks. Future steps include testing on modern production-grade hardware, evaluating performance on higher-bandwidth networks, and extending the setup to heterogeneous systems (CPU+GPU). Overall, significant potential for further development exists, and the primary objective of automating the cluster setup was successfully achieved.

References

[1] Install Guide (v3.2): Rocky 9.4/x86 64 + Warewulf4 + SLURM OpenHPC, a Linux Foundation Collaborative Project, https://github.com/openhpc/ohpc/releases/tag/v3.2.GA
[2] Ansible community documentation, https://docs.ansible.com/ansible/latest/index.html
[3] Open OnDemand documentation, https://docs.rcd.clemson.edu/openod/
[4] BatchConnect VSCode server configurations, https://github.com/OSC/bc_osc_codeserver.git
[5] Intel MPI Benchmarks, https://www.intel.com/content/www/us/en/docs/mpi-library/user-guide-benchmarks/2021-2/overview.html
[6] Cisco Packet Tracer https://prelogin-authoring.netacad.com/courses/packet-tracer